



Last updated on **May 23, 2024**

Structured Streaming patterns on Databricks

This contains notebooks and code samples for common patterns for working with Structured Streaming on Databricks.

Getting started with Structured Streaming

If you are brand new to Structured Streaming, see [Run your first Structured Streaming workload](#).

Write to Cassandra as a sink for Structured Streaming in Python

[Apache Cassandra](#) is a distributed, low-latency, scalable, highly-available OLTP database.

Structured Streaming works with Cassandra through the [Spark Cassandra Connector](#). This connector supports both RDD and DataFrame APIs, and it has native support for writing streaming data. **Important** You must use the corresponding version of the [spark-cassandra-connector-assembly](#).

The following example connects to one or more hosts in a Cassandra database cluster. It also specifies connection configurations such as the checkpoint location and the specific keyspace and table names:

Python

```
spark.conf.set("spark.cassandra.connection.host", "host1,host2")

df.writeStream \
    .format("org.apache.spark.sql.cassandra") \
```

 Ask Assistant

```
.outputMode("append") \
.option("checkpointLocation", "/path/to/checkpoint") \
.option("keyspace", "keyspace_name") \
.option("table", "table_name") \
.start()
```

Write to Azure Synapse Analytics using `foreachBatch()` in Python

`streamingDF.writeStream.foreachBatch()` allows you to reuse existing batch data writers to write the output of a streaming query to Azure Synapse Analytics. See the [foreachBatch documentation](#) for details.

To run this example, you need the Azure Synapse Analytics connector. For details on the Azure Synapse Analytics connector, see [Query data in Azure Synapse Analytics](#).

Python

```
from pyspark.sql.functions import *
from pyspark.sql import *

def writeToSQLWarehouse(df, epochId):
    df.write \
        .format("com.databricks.spark.sqldw") \
        .mode('overwrite') \
        .option("url", "jdbc:sqlserver://<the-rest-of-the-connection-string>") \
        .option("forward_spark_azure_storage_credentials", "true") \
        .option("dbtable", "my_table_in_dw_copy") \
        .option("tempdir", "wasbs://<your-container-name>@<your-storage-account-name>.blob.core.windows.net/<your-directory-name>") \
        .save()

spark.conf.set("spark.sql.shuffle.partitions", "1")

query = (
    spark.readStream.format("rate").load()
        .selectExpr("value % 10 as key")
        .groupBy("key")
        .count()
        .toDF("key", "count")
        .writeStream
        .foreachBatch(writeToSQLWarehouse)
```

```
.outputMode("update")
.start()
)
```

Write to Amazon DynamoDB using `foreach()` in Scala and Python

`streamingDF.writeStream.foreach()` allows you to write the output of a streaming query to arbitrary locations.

Use Python

This example shows how to use `streamingDataFrame.writeStream.foreach()` in Python to write to DynamoDB. The first step gets the DynamoDB boto resource. This example is written to use `access_key` and `secret_key`, but Databricks recommends that you use instance profiles. See [Tutorial: Configure S3 access with an instance profile](#).

1. Define a few helper methods to create DynamoDB table for running the example.

Python

```
table_name = "PythonForeachTest"

def get_dynamodb():
    import boto3

    access_key = "<access key>"
    secret_key = "<secret key>"
    region = "<region name>"
    return boto3.resource('dynamodb',
                           aws_access_key_id=access_key,
                           aws_secret_access_key=secret_key,
                           region_name=region)

def createTableIfNotExists():
    """
    Create a DynamoDB table if it does not exist.
    This must be run on the Spark driver, and not inside foreach.
    """
    dynamodb = get_dynamodb()
```

 Ask Assistant

```

existing_tables = dynamodb.meta.client.list_tables()['TableNames']
if table_name not in existing_tables:
    print("Creating table %s" % table_name)
    table = dynamodb.create_table(
        TableName=table_name,
        KeySchema=[ { 'AttributeName': 'key', 'KeyType': 'HASH' } ],
        AttributeDefinitions=[ { 'AttributeName': 'key', 'AttributeType': 'S' } ],
        ProvisionedThroughput = { 'ReadCapacityUnits': 5, 'WriteCapacityUnits': 5 }
    )

    print("Waiting for table to be ready")

table.meta.client.get_waiter('table_exists').wait(TableName=table_name)

```

2. Define the classes and methods that writes to DynamoDB and then call them from `foreach`. There are two ways to specify your custom logic in `foreach`.

- Use a function: This is the simple approach that can be used to write 1 row at a time. However, client/connection initialization to write a row will be done in every call.

Python

```

def sendToDynamoDB_simple(row):
    """
    Function to send a row to DynamoDB.
    When used with `foreach`, this method is going to be called in the executor
    with the generated output rows.
    """
    # Create client object in the executor,
    # do not use client objects created in the driver
    dynamodb = get_dynamodb()

    dynamodb.Table(table_name).put_item(
        Item = { 'key': str(row['key']), 'count': row['count'] })

```

- Use a class with `open`, `process`, and `close` methods: This allows for a more efficient implementation where a client/connection is initialized and multiple rows can be written out.

Python

```

class SendToDynamoDB_ForeachWriter:
    """
    Class to send a set of rows to DynamoDB.
    When used with `foreach`, copies of this class is going to be used to write
    multiple rows in the executor. See the python docs for
    `DataStreamWriter.foreach`
    for more details.
    """

    def open(self, partition_id, epoch_id):
        # This is called first when preparing to send multiple rows.
        # Put all the initialization code inside open() so that a fresh
        # copy of this class is initialized in the executor where open()
        # will be called.
        self.dynamodb = get_dynamodb()
        return True

    def process(self, row):
        # This is called for each row after open() has been called.
        # This implementation sends one row at a time.
        # For further enhancements, contact the Spark+DynamoDB connector
        # team: https://github.com/audienceproject/spark-dynamodb
        self.dynamodb.Table(table_name).put_item(
            Item = { 'key': str(row['key']), 'count': row['count'] })

    def close(self, err):
        # This is called after all the rows have been processed.
        if err:
            raise err

```

3. Invoke `foreach` in your streaming query with the above function or object.

Python

```

from pyspark.sql.functions import *

spark.conf.set("spark.sql.shuffle.partitions", "1")

query = (
    spark.readStream.format("rate").load()
    .selectExpr("value % 10 as key")
    .groupBy("key")
    .count()
    .toDF("key", "count")
    .writeStream
    .foreach(SendToDynamoDB_ForeachWriter())

```

 Ask Assistant

```

    #.foreach(sendToDynamoDB_simple) // alternative, use one or the other
    .outputMode("update")
    .start()
)

```

Use Scala

This example shows how to use `streamingDataFrame.writeStream.foreach()` in Scala to write to DynamoDB.

To run this you will have to create a DynamoDB table that has a single string key named "value".

1. Define an implementation of the `ForeachWriter` interface that performs the write.

Scala

```

import org.apache.spark.sql.{ForeachWriter, Row}
import com.amazonaws.AmazonServiceException
import com.amazonaws.auth._
import com.amazonaws.services.dynamodbv2.AmazonDynamoDB
import com.amazonaws.services.dynamodbv2.AmazonDynamoDBClientBuilder
import com.amazonaws.services.dynamodbv2.model.AttributeValue
import com.amazonaws.services.dynamodbv2.model.ResourceNotFoundException
import java.util.ArrayList

import scala.collection.JavaConverters._

class DynamoDbWriter extends ForeachWriter[Row] {
  private val tableName = "<table name>"
  private val accessKey = "<aws access key>"
  private val secretKey = "<aws secret key>"
  private val regionName = "<region>"

  // This will lazily be initialized only when open() is called
  lazy val ddb = AmazonDynamoDBClientBuilder.standard()
    .withCredentials(new AWSStaticCredentialsProvider(new
BasicAWSCredentials(accessKey, secretKey)))
    .withRegion(regionName)
    .build()

  //
  // This is called first when preparing to send multiple rows.
  // Put all the initialization code inside open() so that a fresh
  // copy of this class is initialized in the executor where open()

```

✦ Ask Assistant

```

// will be called.
//
def open(partitionId: Long, epochId: Long) = {
    ddb // force the initialization of the client
    true
}

//
// This is called for each row after open() has been called.
// This implementation sends one row at a time.
// A more efficient implementation can be to send batches of rows at a time.
//
def process(row: Row) = {
    val rowAsMap = row.getValuesMap(row.schema.fieldNames)
    val dynamoItem = rowAsMap.mapValues {
        v: Any => new AttributeValue(v.toString)
    }.asJava

    ddb.putItem(tableName, dynamoItem)
}

//
// This is called after all the rows have been processed.
//
def close(errorOrNull: Throwable) = {
    ddb.shutdown()
}
}

```

2. Use the `DynamoDbWriter` to write a rate stream into DynamoDB.

Scala

```

spark.readStream
    .format("rate")
    .load()
    .select("value")
    .writeStream
    .foreach(new DynamoDbWriter)
    .start()

```

Stream-Stream joins

These two notebooks show how to use stream-stream joins in Python and Scala.

Stream-Stream joins Python notebook

[Open notebook in new tab](#)

 Copy link for import

Stream-Stream Joins using Structured Streaming (Python)

This notebook illustrates different ways of joining streams.

We are going to use the canonical example of ad monetization, where we want to find out which ad impressions led to user clicks. Typically, in such scenarios, there are two streams of data from different sources - ad impressions and ad clicks. Both type of events have a common ad identifier (say, `adId`), and we want to match clicks with impressions

[Expand notebook](#)

`ad`. In addition, each event also has a timestamp, which we will use to

Stream-Stream joins Scala notebook

[Open notebook in new tab](#)

 Copy link for import

Stream-Stream Joins using Structured Streaming (Scala)

This notebook illustrates different ways of joining streams.

We are going to use the canonical example of ad monetization, where we want to find out which ad impressions led to user clicks. Typically, in such scenarios, there are two streams of data from different sources - ad impressions and ad clicks. Both type of events have a common ad identifier (say, `adId`), and we want to match clicks with impressions

[Expand notebook](#)

`ad`. In addition, each event also has a timestamp, which we will use to